

SMART CONTRACT AUDIT REPORT

for

United Stables (U)

Prepared By: Xiaomi Huang

PeckShield

March 3, 2026

Document Properties

Client	United Stables
Title	Smart Contract Audit Report
Target	United Stables
Version	1.0
Author	Xuxian Jiang
Auditors	Matthew Jiang, Xuxian Jiang
Reviewed by	Xiaomi Huang
Approved by	Xuxian Jiang
Classification	Public

Version Info

Version	Date	Author(s)	Description
1.0	March 3, 2026	Xuxian Jiang	Final Release

Contact

For more information about this document and its contents, please contact PeckShield Inc.

Name	Xiaomi Huang
Phone	+86 156 0639 2692
Email	contact@peckshield.com

Contents

1 Introduction 4
- 1.1 About United Stables . . . 4
- 1.2 About PeckShield . . . 5
- 1.3 Methodology . . . 5
- 1.4 Disclaimer . . . 7

2 Findings 8
- 2.1 Summary . . . 8
- 2.2 Key Findings . . . 9

3 ERC20 Compliance Checks 10

4 Detailed Results 13
- 4.1 Trust Issue of Admin Keys . . . 13

5 Conclusion 16

References 17

PeckShield

1 Introduction

Given the opportunity to review the design document and related source code of the United Stables token contract, we outline in the report our systematic method to evaluate potential security issues in the smart contract implementation, expose possible semantic inconsistency between smart contract code and the documentation, and provide additional suggestions or recommendations for improvement. Our results show that the given version of the smart contract has no ERC20-compliance issues, though it can be further improved by addressing the identified issues. This document outlines our audit results.

1.1 About United Stables

United Stables is an upgradeable ERC20 stablecoin built on the OpenZeppelin standard, featuring pause capability, freezing functionality, and gas-less transaction support. It is designed to conform to EIP-20, EIP-712, and EIP-2612. This audit focuses on its ERC20-compliance and security. The basic information of the audited contract is as follows:

Table 1.1: Basic Information of United Stables Token

Item	Description
Name	United Stables
Type	ERC20 Token Contract
Platform	Solidity
Audit Method	Whitebox
Latest Audit Report	March 3, 2026

In the following, we show the Git repository of reviewed files and the commit hash value used in this audit.

- https://github.com/United-Coin-U/U.git (0d8d9c2)

1.2 About PeckShield

PeckShield Inc. [4] is a leading blockchain security company with the goal of elevating the security, privacy, and usability of current blockchain ecosystem by offering top-notch, industry-leading services and products (including the service of smart contract auditing). We are reachable at Telegram (https://t.me/peckshield), Twitter (http://twitter.com/peckshield), or Email (contact@peckshield.com).

1.3 Methodology

To standardize the evaluation, we define the following terminology based on OWASP Risk Rating Methodology [3]:

- Likelihood represents how likely a particular vulnerability is to be uncovered and exploited in the wild;
- Impact measures the technical loss and business damage of a successful attack;
- Severity demonstrates the overall criticality of the risk;

Likelihood and impact are categorized into three ratings: *H*, *M* and *L*, i.e., *high*, *medium* and *low* respectively. Severity is determined by likelihood and impact and can be classified into four categories accordingly, i.e., *Critical*, *High*, *Medium*, *Low* shown in Table 1.2.

Table 1.2: Vulnerability Severity Classification

Impact \ Likelihood	*High*	*Medium*	*Low*
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

We perform the audit according to the following procedures:

- Basic Coding Bugs: We first statically analyze given smart contracts with our proprietary static code analyzer for known coding bugs, and then manually verify (reject or confirm) all the issues found by our tool.

- ERC20 Compliance Checks: We then manually check whether the implementation logic of the audited smart contract(s) follows the standard ERC20 specification and other best practices.
- Additional Recommendations: We also provide additional suggestions regarding the coding and development of smart contracts from the perspective of proven programming practices.

Table 1.3: The Full List of Check Items

Category	Check Item
Basic Coding Bugs	Constructor Mismatch
	Ownership Takeover
	Redundant Fallback Function
	Overflows & Underflows
	Reentrancy
	Money-Giving Bug
	Blackhole
	Unauthorized Self-Destruct
	Revert DoS
	Unchecked External Call
	Gasless Send
	Send Instead of Transfer
	Costly Loop
	(Unsafe) Use of Untrusted Libraries
	(Unsafe) Use of Predictable Variables
	Transaction Ordering Dependence
	Deprecated Uses
	Approve / TransferFrom Race Condition
ERC20 Compliance Checks	Compliance Checks (Section 3)
Additional Recommendations	Avoiding Use of Variadic Byte Array
	Using Fixed Compiler Version
	Making Visibility Level Explicit
	Making Type Inference Explicit
	Adhering To Function Declaration Strictly
	Following Other Best Practices

To evaluate the risk, we go through a list of check items and each would be labeled with a severity category. For one check item, if our tool does not identify any issue, the contract is considered safe regarding the check item. For any discovered issue, we might further deploy contracts on our private testnet and run tests to confirm the findings. If necessary, we would additionally build a PoC to demonstrate the possibility of exploitation. The concrete list of check items is shown in Table 1.3.

1.4 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of the given smart contract(s) or blockchain software, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, we always recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contract(s). Last but not least, this security audit should not be used as investment advice.

PeckShield

2 | Findings

2.1 Summary

Here is a summary of our findings after analyzing the `United Stables` token contract. During the first phase of our audit, we study the smart contract source code and run our in-house static code analyzer through the codebase. The purpose here is to statically identify known coding bugs, and then manually verify (reject or confirm) issues reported by our tool. We further manually review business logics, examine system operations, and place ERC20-related aspects under scrutiny to uncover possible pitfalls and/or bugs.

Severity	# of Findings	
Critical	0	
High	0	
Medium	1	■
Low	0	
Informational	0	
Total	1	

Moreover, we explicitly evaluate whether the given contracts follow the standard ERC20 specification and other known best practices, and validate its compatibility with other similar ERC20 tokens and current DeFi protocols. The detailed ERC20 compliance checks are reported in Section 3. After that, we examine a few identified issues of varying severities that need to be brought up and paid more attention to. (The findings are categorized in the above table.) Additional information can be found in the next subsection, and the detailed discussions are in Section 4.

2.2 Key Findings

Overall, no ERC20 compliance issue was found and our detailed checklist can be found in Section 3. While there is no critical or high severity issue, the implementation can be improved by resolving the identified issues (shown in Table 2.1), including 1 medium-severity vulnerability.

Table 2.1: Key United Stables Audit Findings

ID	Severity	Title	Category	Status
PVE-001	Medium	Trust Issue Of Admin Keys	Security Features	Mitigated

Besides recommending specific countermeasures to mitigate the above issue(s), we also emphasize that it is always important to develop necessary risk-control mechanisms and make contingency plans, which may need to be exercised before the mainnet deployment. The risk-control mechanisms need to kick in at the very moment when the contracts are being deployed in mainnet. Please refer to Section 3 for our detailed compliance checks and Section 4 for elaboration of reported issues.

PeckShield

3 | ERC20 Compliance Checks

The ERC20 specification defines a list of API functions (and relevant events) that each token contract is expected to implement (and emit). The failure to meet these requirements means the token contract cannot be considered to be ERC20-compliant. Naturally, as the first step of our audit, we examine the list of API functions defined by the ERC20 specification and validate whether there exist any inconsistency or incompatibility in the implementation or the inherent business logic of the audited contract(s).

Table 3.1: Basic `View-Only` Functions Defined in The ERC20 Specification

Item	Description	Status
name()	Is declared as a public view function	✓
	Returns a string, for example "Tether USD"	✓
symbol()	Is declared as a public view function	✓
	Returns the symbol by which the token contract should be known, for example "USDT". It is usually 3 or 4 characters in length	✓
decimals()	Is declared as a public view function	✓
	Returns decimals, which refers to how divisible a token can be, from 0 (not at all divisible) to 18 (pretty much continuous) and even higher if required	✓
totalSupply()	Is declared as a public view function	✓
	Returns the number of total supplied tokens, including the total minted tokens (minus the total burned tokens) ever since the deployment	✓
balanceOf()	Is declared as a public view function	✓
	Anyone can query any address' balance, as all data on the blockchain is public	✓
allowance()	Is declared as a public view function	✓
	Returns the amount which the spender is still allowed to withdraw from the owner	✓

Our analysis shows that there is no ERC20 inconsistency or incompatibility issue found in the audited `United Stables` token contract. In the surrounding two tables, we outline the respective list of basic `view-only` functions (Table 3.1) and key `state-changing` functions (Table 3.2) according to the widely-adopted ERC20 specification.

Table 3.2: Key `State-Changing` Functions Defined in The ERC20 Specification

Item	Description	Status
transfer()	Is declared as a public function	✓
	Returns a boolean value which accurately reflects the token transfer status	✓
	Reverts if the caller does not have enough tokens to spend	✓
	Allows zero amount transfers	✓
	Emits Transfer() event when tokens are transferred successfully (include 0 amount transfers)	✓
	Reverts while transferring to zero address	✓
transferFrom()	Is declared as a public function	✓
	Returns a boolean value which accurately reflects the token transfer status	✓
	Reverts if the spender does not have enough token allowances to spend	✓
	Updates the spender's token allowances when tokens are transferred successfully	✓
	Reverts if the from address does not have enough tokens to spend	✓
	Allows zero amount transfers	✓
	Emits Transfer() event when tokens are transferred successfully (include 0 amount transfers)	✓
	Reverts while transferring from zero address	✓
	Reverts while transferring to zero address	✓
approve()	Is declared as a public function	✓
	Returns a boolean value which accurately reflects the token approval status	✓
	Emits Approval() event when tokens are approved successfully	✓
	Reverts while approving to zero address	✓
Transfer() event	Is emitted when tokens are transferred, including zero value transfers	✓
	Is emitted with the from address set to $address(0x0)$ when new tokens are generated	✓
Approval() event	Is emitted on any successful call to approve()	✓

In addition, we perform a further examination on certain features that are permitted by the ERC20 specification or even further extended in follow-up refinements and enhancements, but not required for implementation. These features are generally helpful, but may also impact or bring certain incompatibility with current DeFi protocols. Therefore, we consider it is important to highlight them as well. This list is shown in Table 3.3.

Table 3.3: Additional Opt-in Features Examined in Our Audit

Feature	Description	Opt-in
Deflationary	Part of the tokens are burned or transferred as fee while on transfer()/transferFrom() calls	—
Rebasing	The balanceOf() function returns a re-based balance instead of the actual stored amount of tokens owned by the specific address	—
Pausable	The token contract allows the owner or privileged users to pause the token transfers and other operations	✓
Upgradable	The token contract allows for future upgrades	✓
Whitelistable	The token contract allows the owner or privileged users to whitelist a specific address such that only token transfers and other operations related to that address are allowed	✓
Mintable	The token contract allows the owner or privileged users to mint tokens to a specific address	✓
Burnable	The token contract allows the owner or privileged users to burn tokens of a specific address	✓

4 Detailed Results

4.1 Trust Issue of Admin Keys

- ID: PVE-001
- Severity: Medium
- Likelihood: Medium
- Impact: Medium
- Target: `Stablecoin`
- Category: Security Features [2]
- CWE subcategory: CWE-287 [1]

Description

In the `United Stables` token contract, there is a privileged `owner` account that plays a critical role in governing and regulating the token-wide operations (e.g., mint tokens, block accounts, pause transfers, and upgrade token contract). Our analysis shows that the privileged account needs to be scrutinized. In the following, we examine the privileged account and the related privileged accesses in current contract.

```
102     function setAutoMintMaxLimit(uint256 limit) external onlyOwner {
103         emit SetAutoMintMaxLimit(autoMintMaxLimit, limit);
104         autoMintMaxLimit = limit;
105     }
106     ...
107     function mint(uint256 amount) external onlyOwner returns (bool) {
108         _mint(_msgSender(), amount);
109         emit Mint(_msgSender(), _msgSender(), amount);
110         return true;
111     }
112     ...
113     function mint(address to, uint256 amount) external whenNotPaused notFrozen(to)
            onlyOwner returns (bool) {
114         _mint(to, amount);
115         emit Mint(_msgSender(), to, amount);
116         return true;
117     }
118     ...
```

```
119     function autoMint(address to, uint256 amount, uint256 seq, uint256 chain) external
            whenNotPaused notFrozen(to) onlyAutoOwner returns (bool) {
120         require(seq == nonce, InvalidNonce(seq));
121         require(chain == chainId, InvalidChainId(chain));
122         require(amount > 0, InvalidAmount(amount));
123         require(autoMintMaxLimit >= amount, MintLimitExceeded(amount, autoMintMaxLimit))
                ;
124         nonce++;
125         _mint(to, amount);
126         emit Mint(_msgSender(), to, amount);
127         emit AutoMint(_msgSender(), to, seq, amount);
128         return true;
129     }
130     ...
131     function burn(uint256 amount) external onlyOwner returns (bool) {
132         _burn(_msgSender(), amount);
133         emit Burn(_msgSender(), _msgSender(), amount);
134         return true;
135     }
136     ...
137     function autoBurn(uint256 amount, uint256 seq, uint256 chain) external whenNotPaused
             onlyAutoOwner returns (bool) {
138        require(seq == nonce, InvalidNonce(seq));
139         require(chain == chainId, InvalidChainId(chain));
140         require(amount > 0, InvalidAmount(amount));
141         nonce++;
142         address owner = owner();
143         _burn(owner, amount);
144         emit Burn(autoOwner, owner, amount);
145         emit AutoBurn(autoOwner,owner, seq, amount);
146         return true;
147     }
148     ...
149     function freeze(address account) external onlyOwner {
150         frozen[account] = true;
151         emit Freeze(_msgSender(), account);
152     }
153     ...
154     function unfreeze(address account) external onlyOwner {
155         delete frozen[account];
156         emit Unfreeze(_msgSender(), account);
157     }
158     ...
159     function pause() external onlyOwner {
160         _pause();
161     }
162     ...
163     function unpause() external onlyOwner {
164         _unpause();
165     }
```

Listing 4.1: Example Privileged Operations in `United Stables`

We emphasize that the privilege assignment may be necessary and consistent with the token design. However, it would be worrisome if the privileged account is a plain EOA account. Note that a multi-sig account could greatly alleviate this concern, though it is still far from perfect. Specifically, a better approach is to eliminate the administration key concern by transferring the role to a community-governed DAO. In the meantime, a timelock-based mechanism can also be considered as mitigation.

In the meantime, the audited token contract makes use of the proxy contract to allow for future upgrades. The upgrade is a privileged operation and the management of the related admin key also falls in this trust issue.

Recommendation Promptly transfer the privileged account to the intended DAO-like governance contract. All changed to privileged operations may need to be mediated with necessary timelocks. Eventually, activate the normal on-chain community-based governance life-cycle and ensure the intended trustless nature and high-quality distributed governance.

Status This issue has been mitigated as the team plans to use a multi-sig as the `owner` account.

PeckShield

5 | Conclusion

In this security audit, we have examined the `United Stables` token design and implementation. During our audit, we first checked all respects related to the compatibility of the `ERC20` specification and other known `ERC20` pitfalls/vulnerabilities. We then proceeded to examine other areas such as coding practices and business logics. Overall, no issues of concern or vulnerabilities were discovered. In the meantime, as disclaimed in Section 1.4, we appreciate any constructive feedbacks or suggestions about our findings, procedures, audit scope, etc.

PeckShield

References

[1] MITRE. CWE-287: Improper Authentication. https://cwe.mitre.org/data/definitions/287.html.

[2] MITRE. CWE CATEGORY: 7PK - Security Features. https://cwe.mitre.org/data/definitions/254.html.

[3] OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology.

[4] PeckShield. PeckShield Inc. https://www.peckshield.com.

PeckShield